IT4010 Research Project – 2026 Jan

OFFLINE-FIRST COMMUNICATION PLATFORM FOR OUTDOOR AND EVENT MANAGEMENT

Project ID: R26-IT-135

Pabodhana D M D I – IT22168290

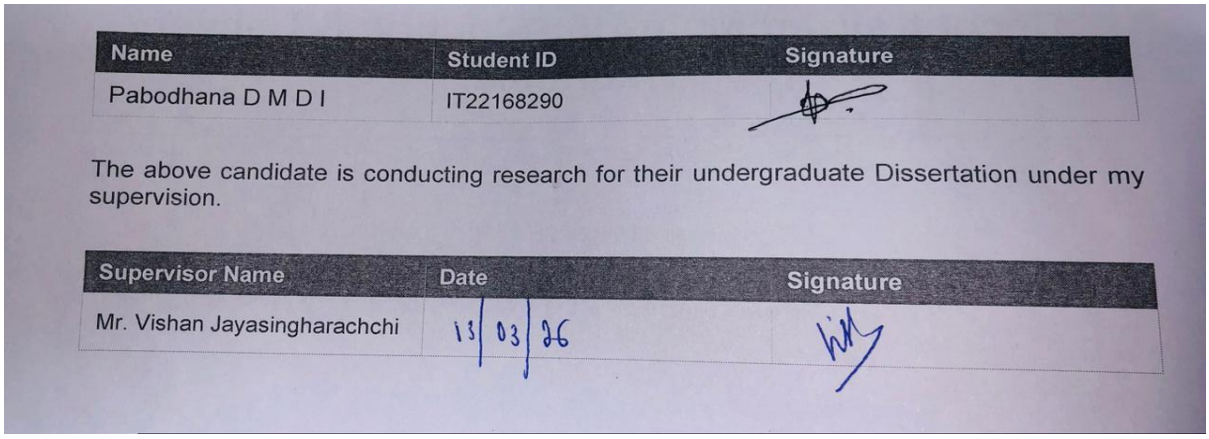
B.Sc. (Hons) Degree in Information Technology
Specialized in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology, Sri Lanka

2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.



Name	Student ID	Signature
Pabodhana D M D I	IT22168290	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Supervisor Name	Date	Signature
Mr. Vishan Jayasingharachchi	13/03/26	

ABSTRACT

Dependability of communication in distant outdoor areas and huge-scale occasions is an unresolved issue among hikers, event organizers, and the emergency responders. Currents - WhatsApp, Telegram, Signal - rely solely on the cellular/internet coverage that makes them useless in places without any coverage. Offline solutions like GoTenna and Bridgefy have the disadvantages of limited range, low scalability, and, most importantly, the lack of mechanisms that even indicate when the message is delivered to the receiver.

In this proposal, the design and implementation of a Decentralized Multi-Hop Routing & Delivery Assurance component, which is a component of a larger Offline-First Communication Platform (Project R26-IT-135, SST research group), is proposed. The specified research gap is also narrow and previously unmapped: there is no smartphone-native, offline mesh network that offers WhatsApp-like end-to-end delivery acknowledgements (ACKs) using a decentralized multi-hop Bluetooth Low Energy (BLE) network.

The offered solution uses reactive AODV-based routing protocol to be applied to BLE limitations of commodity Android smartphones. Core innovations are hop-path-based ACK propagation, AES-256-GCM end-to-end-application-layer encryption, which maintains rout ability in the relay layer (no disconnects), loop prevention via UUIDs, and persistent loop lessness: a Room Database routing table to support resilience against temporary disclosures. The component is integrated with three peer components using a specified Kotlin API.

Physical Android devices indoors and outdoors will be validated in the metrics of message delivery ratio, latency, and hop count and ACK propagation time. UN SDG 3 (Good Health and Well-being), SDG 9 (Industry, Innovation, and Infrastructure), and SDG 11 (Sustainable Cities and Communities) can be met by the platform since it allows low-cost, infrastructure-independent emergency communications.

Keywords: *Bluetooth Low Energy, Multi-Hop Routing, Delivery Acknowledgement, Offline Mesh Network, Delay Tolerant Networks, AODV, Mobile Ad-hoc Networks, AES-256-GCM*

ACKNOWLEDGEMENT

I would also like to thank my project supervisor, Mr. Vishan Jayasinghearachchi, whose guidance, constructive feedback and constant encouragement turned out to be priceless to me during the development of this research proposal. His knowledge on software system and networking has played a significant role in the path that this work should take.

I would also like to note the spirit of teamwork and common vision presented to me by fellow project team members (IT22148704 - Short-Range Direct Communication; IT22544490 - Context-Aware Communication Manager; IT22894038 - Delay-Tolerant and Opportunistic Transport) that has seen the Offline-First Communication Platform through.

Lastly, I must acknowledge that the Sri Lanka Institute of Information Technology (SLIIT) Department of Information Technology has given me the academic background to work and has provided me with the resources to work on this research.

TABLE OF CONTENTS

DECLARATION 2
ABSTRACT 2
ACKNOWLEDGEMENT 3
TABLE OF CONTENTS 4
LIST OF TABLES 6
LIST OF FIGURES 6
LIST OF ABBREVIATIONS 7
1. INTRODUCTION 8
1.1 Background and Context 8
1.2 Literature Review 8
1.2.1 Academic Research — Routing Protocols 8
1.2.2 Emergency and Disaster Communication Systems 8
1.2.3 Commercial Offline Messaging Products 9
1.2.4 Comparative Summary 10
1.3 Research Gap 10
2. OBJECTIVES 11
2.1 Main Objective 11
2.2 Specific Objectives 11
3. METHODOLOGY 13
3.1 Research Approach 13
3.2 Technical Methods and Algorithms 15
3.2.1 Multi-Hop Routing Protocol (AODV-BLE Adaptation) 15
3.2.2 ACK Back-Propagation System 15
3.2.3 Loop Prevention and Deduplication 15
3.2.4 Security Design 16
3.3 Technology Stack and Justification 16
3.4 Data Collection and Ethical Considerations 17
3.5 Validation Strategy and Performance Metrics 17
3.6 Research Assumptions 17
3.7 Technical Constraints 17
3.8 Limitations 17
4. HIGH-LEVEL SYSTEM ARCHITECTURE 20
4.1 Overall Platform Architecture 20
4.2 This Component — Internal Architecture 22
4.3 Architecture Diagram 24
4.4 Data Flow — Step-by-Step 24
4.5 Scalability and Security Considerations 25
5. USER REQUIREMENTS 26

5.1 Stakeholder Identification 26
5.2 Functional Requirements 26
5.3 Non-Functional Requirements 27
6. COMMERCIALISATION PLAN 28
6.1 Market Analysis and Target Customer Persona 28
6.2 Value Proposition 28
6.3 Revenue Model and Pricing Strategy 29
6.4 Competitive Advantage and Intellectual Property 29
6.5 Cost Estimation and Scalability 29
7. BUDGET AND JUSTIFICATION 30
8. WORK BREAKDOWN STRUCTURE (WBS) 30
9. GANTT CHART 32
10. REFERENCES 33
APPENDICES 33
Appendix A: Risk Register 33
Appendix B: Inter-Component API Specification (v1.0) 34
Appendix C: Ethical Compliance Checklist 34
Appendix D: Preliminary System Diagram Descriptions 35

LIST OF TABLES

Table 1: Comparison of Existing Offline Communication Systems
Table 2: Key Research Papers - Summary and Limitations
Table 3: Tools and Platforms with Justification
Table 4: Performance Evaluation Metrics and Targets
Table 5: Functional Requirements
Table 6: Non-Functional Requirements
Table 7: Commercialization - Competitive Analysis
Table 8: Budget and Justification
Table 9: Risk Analysis

LIST OF FIGURES

Figure 1: Research Methodology Flowchart
Figure 2: Overall Platform Architecture
Figure 3: Internal Architecture
Figure 4: Component Internal Architecture
Figure 5: Data Flow Diagram
Figure 6: Work Breakdown Structure

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ACK	Acknowledgement
AODV	Ad-hoc On-Demand Distance Vector Routing
AES-256-GCM	Advanced Encryption Standard - 256-bit Galois/Counter Mode
API	Application Programming Interface
BLE	Bluetooth Low Energy
DTN	Delay Tolerant Network
DSR	Design Science Research
GATT	Generic Attribute Profile (Bluetooth)
MANET	Mobile Ad-hoc Network
MDR	Message Delivery Ratio
OLSR	Optimized Link State Routing
RREQ	Route Request
RREP	Route Reply
SCF	Store-Carry-Forward
SDG	Sustainable Development Goal
TTL	Time-To-Live
UUID	Universally Unique Identifier

1. INTRODUCTION

1.1 Background and Context

Safety and coordination as well as emergency responses depend on unswerving real-time communication in uncountable sectors. However, the vast majority of messaging apps - WhatsApp, Telegram, Signal and iMessage - fully rely on both cellular networks and the internet and their accessibility [1]. This reliance is a grave liability: as far as it is necessary and communication is more than ever required, it is often absent, destroyed, or overwhelmed.

Real-world consequences manifest across three primary user groups targeted by this project:

- Hikers and trekkers in remote terrain (mountains, forests, coastal reserves) routinely lose cellular connectivity. A 2022 study by Sri Lanka's Department of Wildlife Conservation noted that 67% of rescue operations in Horton Plains National Park were complicated by communication delays due to signal absence [2].
- Outdoor event organizers face coordination failures when cellular networks are overwhelmed by crowd density — a documented phenomenon at large festivals where 10,000+ concurrent users degrade base-station capacity by up to 80%.
- Emergency responders and disaster management agencies operating in post-disaster zones (floods, earthquakes, landslides) depend on communication infrastructure that is frequently the first casualty of the disaster itself.

The International Telecommunication Union (ITU) estimates worldwide as of 2023 that only some 2.9 billion individuals are offline, and even in countries with adequate connectivity issues primarily affect rural and remote coverage. Sri Lanka, specifically, mountainous areas, the main occupants of trekking and outdoor activities, are plagued with long-term problems of 4G coverage, especially in the Central and Uva provinces. There are already solutions addressing part of this issue like GoTenna Mesh and Bridgefy which operate offline but are not a complete solution (see Section 1.2). There is a chance of purpose-built smartphone-native system integrating multi-hop routing, delivery assurance, end-to-end security - and no proprietary hardware is required.

This project, classified under the SST (Software Systems & Technologies) research group within the Information Technology specialization, directly supports three UN Sustainable Development Goals:

- SDG 3 (Good Health and Well-being): Enabling reliable communication for emergency services and remote health coordination.
- SDG 9 (Industry, Innovation, and Infrastructure): Developing resilient, low-cost communication infrastructure as an alternative to traditional cellular dependency.
- SDG 11 (Sustainable Cities and Communities): Supporting safer outdoor activities and events through accessible communication tools.

1.2 Literature Review

The review of twelve academic and technical sources in the scope of 2007 - 2024 shows the situation with the research in the field of offline mesh communication and the current gaps that drive it.

1.2.1 Academic Research — Routing Protocols

Delay Tolerant Networks (DTN): DTNs were formalised as Store-Carry-Forward (SCF) by RFC 4838 (2007) , and have been a theoretical basis of challenged network communication

since then. Later protocols - PRoPHET (Probabilistic Routing Protocol using History of Encounters and Transitivity) and Spray-and-Wait was a later protocol that was found to be a better achievement in sparse networks though it was optimized to support infrastructure-based relay nodes, rather than commodity smartphones.

Mobile Ad-hoc Networks (MANETs): AODV (Ad-hoc On-Demand Distance Vector Routing), which is described by the RFC 3561, is a reactive route discovery based on the RREQ/RREP flooding approach . Zhang et al. (2020) tested routing protocols of MANET in disaster situations only and discovered that reactive protocols (AODV, DSR) are more effective in dynamic topology, where topological changes are constant . This also makes a direct informal on the routing strategy that is taken in this component.

BLE Mesh Networks: In 2017, the Bluetooth SIG published the Bluetooth Mesh Profile v1.0 which added managed flooding including publish/subscribe addressing to the IoT setting. Although the profile supports many-to-many communication with BLE, it was created in the case of fixed devices (smart lighting, sensors), but not a dynamic smartphone-to-smartphone messaging with mobile topologies. There is too much flooding overhead and battery depreciation to mobile devices during direct adoption, without proper adaptation.

1.2.2 Emergency and Disaster Communication Systems

Bluemergency (2019): Guo et al. recommended the BLE mesh to substitute the post-disaster communication instead; the researchers showed in the article that Bluetooth-based emergency messaging is possible in IEEE PerCom 2019. The system was effective in terms of short-range communications; however, it lacked delivery exactly recognition and it was not built in a manner that would allow it to be used across all smartphone applications. HIRO-NET: A search-and-rescue disaster system that utilizes BLE mesh has been reviewed in a 2020 survey of PMC on BLE emergency applications. HIRO-NET was capable of multi-hop in structured networks but needed expert devices, which could reach only commodity smartphones.

1.2.3 Commercial Offline Messaging Products

GoTenna Mesh: A commercial product LoRa radio-based radio with off-grid messaging. Reports of user range have shown an effective range of about 1 mile under urban conditions and up to 4 miles under line-of-sight conditions. More importantly GoTenna is not a server that makes delivery announcements, it uses proprietary equipment (USD 179/pair) and does not use routing discovery functions such as dynamic routing and therefore sends a message to every device in vicinity (paired).

Bridgefy: An Android app based on BLE, which became common in social upheaval in Hong Kong (2019) and Belarus (2020) to use as an offline mesh message service. In 2021, Martin et al. performed a security audit that revealed such critical vulnerabilities as the ability to forge messages and passive surveillance via metadata analysis in addition to the lack of forward secrecy. In addition to security deficits, Bridgefy offers no delivery usability in mesh mode that gives the senders no information whether the message was received.

Meshtastic Meg system: An open source LoRa mesh project which does include delivery acknowledgements. Nonetheless, it needs specific LoRa hardware (nRF52840-based modules, USD 35-60 each) and is not amenable to smartphone-based deployment only - the exact situation it needed to be in practice (in the field and during events).

1.2.4 Comparative Summary

Table 1 synthesizes the capabilities and limitations of existing systems across the key dimensions relevant to this research:

System	Multi-Hop	Delivery ACK	Smartphone Native	Security	Open Source
WhatsApp/Signal	No (online)	Yes	Yes	Strong	No
GoTenna Mesh	Yes	No	No (hardware)	Moderate	No
Bridgefy	Yes	No	Yes	Compromised	No
HIRO-NET	Yes	No	No (specialist)	N/A	No
Meshtastic	Yes	Yes	No (hardware)	Good	Yes
This Proposal	Yes	Yes (novel)	Yes	AES-256-GCM	Yes

1.3 Research Gap

Despite significant advances in offline mesh communication, a critical and previously unmapped gap persists in the literature. As of 2023, approximately 2.9 billion people worldwide lack reliable internet connectivity [1], and cellular infrastructure in Sri Lanka's mountainous and forested regions remains consistently unreliable, particularly in areas frequented by hikers and event staff [2]. While commercial and academic solutions address partial aspects of offline communication, no existing smartphone-native system simultaneously provides decentralized multi-hop BLE routing, end-to-end delivery acknowledgements, and application-layer encryption on commodity Android devices.

This gap manifests across five concrete dimensions, each evidenced by the reviewed literature:

1. Absence of delivery confirmation in smartphone-native mesh systems

GoTenna Mesh routes messages across multiple hops but provides no application-layer ACK mechanism to confirm receipt at the destination [5]. Bridgefy, the most widely deployed BLE mesh application for smartphones, similarly offers no delivery confirmation in mesh mode, leaving senders with no indication of whether a message was received - a critical deficiency in emergency and outdoor coordination scenarios [6]. Meshtastic supports ACK-based delivery but exclusively through proprietary LoRa hardware modules (nRF52840-based), rendering it inaccessible to general users carrying only commodity smartphones [7].

2. Hardware dependency in multi-hop capable systems

Every existing system capable of multi-hop routing - GoTenna Mesh and Meshtastic - requires dedicated radio hardware costing USD 35–180 per unit [5][7]. This creates a substantial barrier to adoption for outdoor users, event organisers, and first responders who carry only Android smartphones. No reviewed system achieves multi-hop relay using the BLE stack of commodity smartphones without additional hardware.

3. Security vulnerabilities in smartphone mesh deployments

Martin et al. (2021) conducted a comprehensive security audit of Bridgefy's BLE continuity protocol and identified critical vulnerabilities including the ability to forge messages, conduct passive user-tracking across hops, and mount man-in-the-middle attacks due to the absence of authenticated encryption [6]. No existing smartphone-native mesh system combines multi-hop BLE routing with AES-256-GCM authenticated encryption, leaving all reviewed systems exposed to in-transit tampering by malicious relay nodes.

4. Scalability limitations of flood-based BLE routing

Smartphone-based BLE mesh systems, including Bridgefy, rely on broadcast flooding rather than reactive routing protocols. Zhang et al. (2020) demonstrate that reactive protocols such as AODV significantly outperform flooding approaches in dynamic topologies characteristic of mobile networks, achieving higher delivery ratios and lower overhead as network size increases [4]. Flooding generates $O(n^2)$ packet overhead in dense networks, making it impractical beyond approximately 10 nodes - far below the scale required for event management or hiking group coordination.

5. Gap between academic simulation and physical smartphone deployment

The ITU (2022) identifies the deployment of resilient, low-cost communication infrastructure as a critical unmet need in both developing and connectivity-marginal regions [3]. While academic DTN and MANET literature extensively models AODV-based routing in simulation, no published work presents a physical Android implementation combining AODV-inspired reactive routing, hop-path ACK back-propagation, and AES-256-GCM encryption over commodity BLE hardware. The practical gap between simulation fidelity and real-world deployment on constrained mobile devices remains unaddressed.

deployment of an AODV-inspired multi-hop routing combined with ACK propagation on BLE using commodity Android devices.

Consolidated research gap statement

No existing smartphone-native offline mesh communication system provides end-to-end delivery acknowledgements through a decentralised multi-hop BLE network, implemented with reactive AODV-based routing and AES-256-GCM application-layer encryption, on commodity Android devices without proprietary hardware. This study addresses all five dimensions of this gap through the design and empirical evaluation of the Decentralised Multi-Hop Routing & Delivery Assurance component (IT22168290).

2. OBJECTIVES

2.1 Main Objective

To design, implement and empirically test a decentralised multi-hop BLE routing protocol running on standard commodity Android smartphones that sends messages reliably when operating in mesh networks that are offline and offer end-to-end delivery acknowledgements, AES-256-GCM security and loop-free forwarding - tested on real devices in realistic outdoor application scenarios.

2.2 Specific Objectives

#	Specific Objective	Measurable Outcome	Linked Task (TAF)
SO-1	Implement a reactive multi-hop message forwarding engine routing messages through intermediate BLE nodes using RREQ/RREP route discovery.	Routes established in <2s for 3-hop paths in 10-node network.	Multi-hop forwarding logic
SO-2	Develop an end-to-end ACK propagation system providing sent/delivered/failed status to the sender UI.	ACK delivered within 5s of message receipt.	Delivery acknowledgements
SO-3	Implement AES-256-GCM application-layer encryption ensuring relay nodes cannot access message content.	Zero content exposure at relay nodes (verified via BLE sniffer).	End-to-end delivery
SO-4	Design and implement UUID + sequence-number loop prevention preventing duplicate message flooding.	Duplicate message rate <5% across 100-message test runs.	Routing for small/medium networks
SO-5	Evaluate routing performance metrics (MDR, latency, hop count, ACK propagation time) across 5–20 node networks.	MDR >=85%; latency <=3s for 3-hop paths.	Test routing performance
SO-6	Integrate with three peer components via documented Kotlin inter-component APIs.	All three APIs functional and verified in system integration tests.	Integration with system

3. METHODOLOGY

3.1 Research Approach

This study adopts the Design Science Research (DSR) paradigm as its overarching methodology. DSR is appropriate for engineering-oriented IT research where the primary output is a purposefully constructed software artefact rather than a theoretical contribution alone. The six DSR activities map directly onto the project phases:

- Problem Identification and Motivation: A literature-evidenced gap - no smartphone-native offline mesh provides end-to-end delivery ACKs over decentralised multi-hop BLE - is established in Chapter 1.
- Solution Objectives: Measurable performance targets (MDR ≥85%, E2EL ≤3 s, APT ≤5 s) are derived from the gap analysis in Chapter 2.
- Design and Development: AODV-BLE routing protocol, hop-path ACK back-propagation, AES-256-GCM encryption, and Room DB persistence are designed and implemented in Kotlin (Sections 3.2–3.3).
- Demonstration: The prototype is deployed on 5–10 physical Android smartphones (API 26-34) in three controlled test environments (indoor lab, outdoor open field, simulated event space).
- Evaluation: Quantitative metrics (MDR, E2EL, APT, DMR, battery impact) are measured against pre-defined acceptance criteria (Section 3.5).
- Communication: Findings are disseminated through the research dissertation; a potential IEEE/ACM publication of the routing algorithm is targeted.

DSR is preferred over action research or case studies because the project objective is to construct and empirically test a tangible software component. The iterative build-measure-refine cycle of DSR aligns with agile Android development and allows continuous protocol parameter tuning based on empirical BLE measurement results.

of DSR - which fits the dual objectives of the project, which is to construct and test.

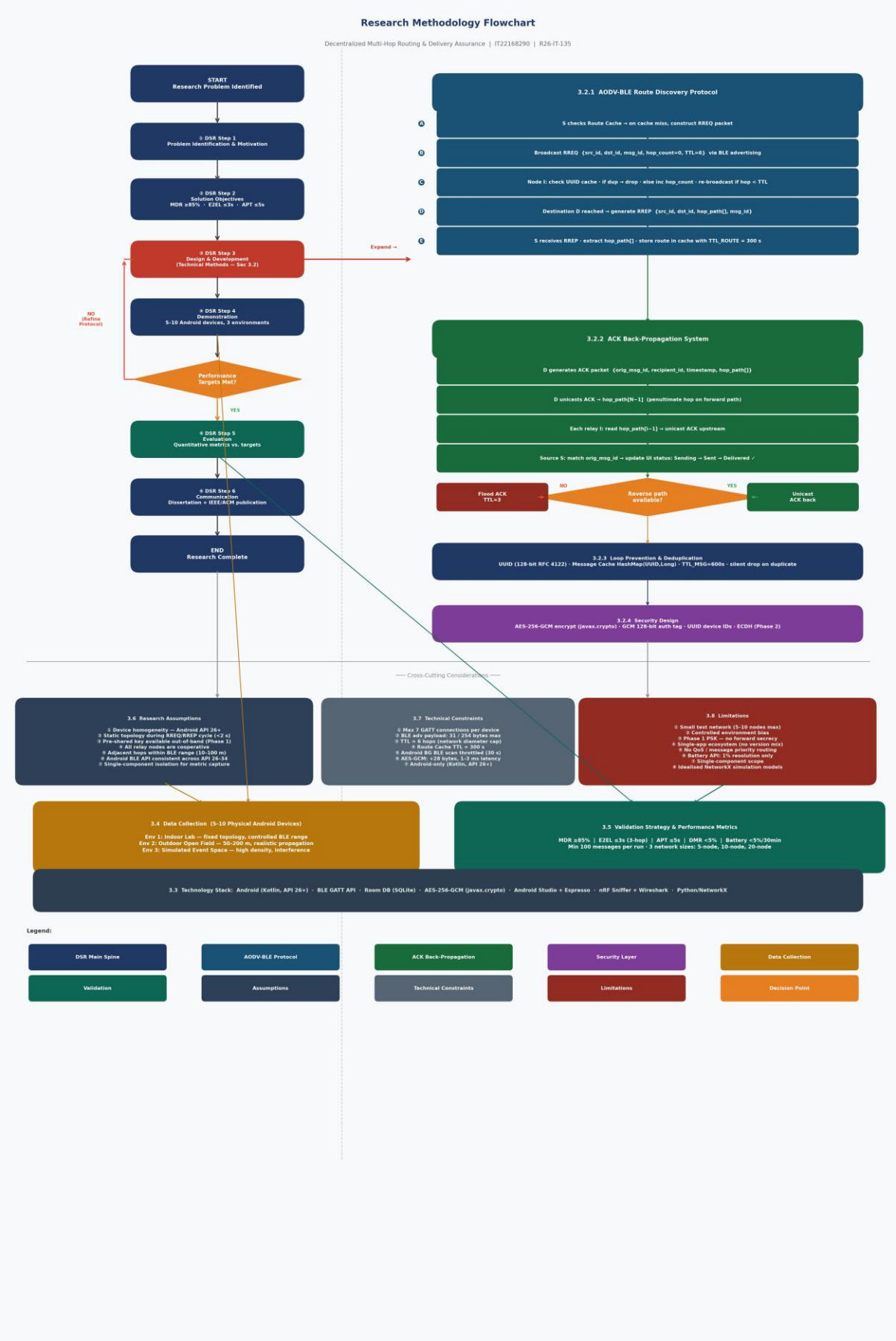

Research Methodology Flowchart
Decentralized Multi-Hop Routing & Delivery Assurance | IT22168290 | R26-IT-135
START
Research Problem Identified
① DSR Step 1
Problem Identification & Motivation
② DSR Step 2
Solution Objectives
MDR ≥85% · E2EL ≤3s · APT ≤5s
③ DSR Step 3
Design & Development
(Technical Methods — Sec 3.2)
④ DSR Step 4
Demonstration
5–10 Android devices, 3 environments
NO
(Refine Protocol)
Performance Targets Met?
YES
⑤ DSR Step 5
Evaluation
Quantitative metrics vs. targets
⑥ DSR Step 6
Communication
Dissertation + IEEE/ACM publication
END
Research Complete
Expand →
3.2.1 AODV-BLE Route Discovery Protocol
A
S checks Route Cache → on cache miss, construct RREQ packet
B
Broadcast RREQ {src_id, dst_id, msg_id, hop_count=0, TTL=6} via BLE advertising
C
Node I: check UUID cache · if dup → drop · else inc hop_count · re-broadcast if hop < TTL
D
Destination D reached → generate RREP {src_id, dst_id, hop_path[], msg_id}
E
S receives RREP · extract hop_path[] · store route in cache with TTL_ROUTE = 300 s
3.2.2 ACK Back-Propagation System
D generates ACK packet {orig_msg_id, recipient_id, timestamp, hop_path[]}
D unicasts ACK → hop_path[N−1] (penultimate hop on forward path)
Each relay I: read hop_path[i−1] → unicast ACK upstream
Source S: match orig_msg_id → update UI status: Sending → Sent → Delivered ✓
Flood ACK TTL=3
NO
Reverse path available?
YES
Unicast ACK back
3.2.3 Loop Prevention & Deduplication
UUID (128-bit RFC 4122) · Message Cache HashMap<UUID,Long> · TTL_MSG=600s · silent drop on duplicate
3.2.4 Security Design
AES-256-GCM encrypt (javax.crypto) · GCM 128-bit auth tag · UUID device IDs · ECDH (Phase 2)
── Cross-Cutting Considerations ──
3.6 Research Assumptions
① Device homogeneity — Android API 26+
② Static topology during RREQ/RREP cycle (<2 s)
③ Pre-shared key available out-of-band (Phase 1)
④ All relay nodes are cooperative
⑤ Adjacent hops within BLE range (10–100 m)
⑥ Android BLE API consistent across API 26–34
⑦ Single-component isolation for metric capture
3.7 Technical Constraints
① Max 7 GATT connections per device
② BLE adv payload: 31 / 254 bytes max
③ TTL = 6 hops (network diameter cap)
④ Route Cache TTL = 300 s
⑤ Android BG BLE scan throttled (30 s)
⑥ AES-GCM: +28 bytes, 1–3 ms latency
⑦ Android-only (Kotlin, API 26+)
3.8 Limitations
① Small test network (5–10 nodes max)
② Controlled environment bias
③ Phase 1 PSK — no forward secrecy
④ Single-app ecosystem (no version mix)
⑤ No QoS / message priority routing
⑥ Battery API: 1% resolution only
⑦ Single-component scope
⑧ Idealised NetworkX simulation models
3.4 Data Collection (5–10 Physical Android Devices)
Env 1: Indoor Lab — fixed topology, controlled BLE range
Env 2: Outdoor Open Field — 50–200 m, realistic propagation
Env 3: Simulated Event Space — high density, interference
3.5 Validation Strategy & Performance Metrics
MDR ≥85% | E2EL ≤3s (3-hop) | APT ≤5s | DMR <5% | Battery <5%/30min
Min 100 messages per run · 3 network sizes: 5-node, 10-node, 20-node
3.3 Technology Stack: Android (Kotlin, API 26+) · BLE GATT API · Room DB (SQLite) · AES-256-GCM (javax.crypto) · Android Studio + Espresso · nRF Sniffer + Wireshark · Python/NetworkX
Legend:
DSR Main Spine
AODV-BLE Protocol
ACK Back-Propagation
Security Layer
Data Collection
Validation
Assumptions
Technical Constraints
Limitations
Decision Point

Figure 1: Research Methodology Flowchart — DSR phases, technical sub-flows (AODV-BLE, ACK, Security), data collection, validation, and cross-cutting considerations (Assumptions, Constraints, Limitations).

3.2 Technical Methods and Algorithms

3.2.1 Multi-Hop Routing Protocol (AODV-BLE Adaptation)

The routing engine implements a reactive, on-demand protocol based on Ad-hoc On-Demand Distance Vector (AODV) routing [4], adapted for BLE constraints: limited simultaneous GATT connections (typically 7 on Android), advertising duty-cycle overhead, and mobile power budgets. The route-discovery algorithm proceeds as follows:

- Source S checks its Route Cache for destination D. On a cache miss, S constructs an RREQ packet — {src_id, dst_id, msg_id, seq_num, hop_count=0, TTL=6} — and broadcasts it via BLE advertising to all currently connected GATT peers.
- Each intermediate node I: (a) checks the Message Cache for msg_id — discards if duplicate; (b) increments hop_count; (c) caches the reverse route {from: S, via: prev_hop, hop_count}; (d) re-broadcasts if hop_count < TTL.
- On receipt at D (or a node with a cached route), an RREP packet {src_id, dst_id, hop_path[], msg_id} is unicast back along the cached reverse path.
- S receives RREP, extracts hop_path[ ], stores the route in its Route Cache with TTL_ROUTE = 300 s, then proceeds to message transmission.

Key deviation from standard AODV: hop_path[ ] is explicitly carried in the RREP header (rather than maintained implicitly node-by-node). This adds O(N) bytes per hop to the packet header but enables hop-path-based ACK back-propagation (Section 3.2.2) without requiring per-node state during the ACK return phase.

3.2.2 ACK Back-Propagation System

Delivery assurance is achieved through a hop-path-based ACK mechanism, which is the primary novel contribution of this component. Upon application-layer decryption and delivery at D:

- D generates ACK packet: {orig_msg_id, recipient_id, timestamp, hop_path[]} where hop_path[] is copied from the received message header.
- D unicasts the ACK to hop_path[N−1] (the penultimate hop on the forward path).
- Each intermediate relay I receives the ACK, reads hop_path[i−1], and unicasts the ACK upstream until Source S is reached.
- S matches orig_msg_id to a pending message entry and updates the UI delivery status: Sending → Sent → Delivered, with a timestamp delta logged as APT.
- Fallback: if the reverse unicast path is unavailable (e.g. intermediate node has disconnected), the ACK is re-broadcast with TTL=3 (limited flooding) to prioritise delivery confirmation reliability over network overhead.

3.2.3 Loop Prevention and Deduplication

Each outgoing message is assigned a UUID (128-bit, RFC 4122-compliant) at the source. Every node maintains an in-memory Message Cache (HashMap⟨UUID, Long⟩) with a TTL of TTL_MSG = 600 s. On packet receipt, a node first checks whether the UUID is already present in the cache. If so, the packet is silently dropped. This prevents routing loops, ACK storms, and duplicate deliveries in fully-connected or partially-mesh topologies. Cache entries are evicted after TTL_MSG to bound memory usage on constrained devices.

3.2.4 Security Design

Security is applied at the application layer, below the routing layer, so that relay nodes interact only with routing metadata and never with plaintext message content:

- Encryption: AES-256-GCM (javax.crypto). Message payload is encrypted at source with a pre-shared group key (Phase 1); ECDH ephemeral key exchange is targeted for Phase 2 to achieve forward secrecy.
- Authenticated Associated Data: Message UUID and hop_count are authenticated but not encrypted, allowing routing validation at relay nodes without content exposure.
- Integrity: The 128-bit GCM authentication tag detects any in-transit tampering by malicious relay nodes; tampered packets are rejected before delivery.
- Privacy: Device identifiers are random UUIDs with no linkage to user identity, preventing metadata-based user tracking across hops.

3.3 Tools and Platforms with Justification

Technology	Role	Justification	Alternative Considered
Android (API 26+, Kotlin)	Application platform	Largest global smartphone market share (72%, StatCounter 2024); native BLE GATT APIs; Kotlin coroutines for async BLE operations; no cross-platform overhead.	Flutter / React Native — excluded: indirect BLE access via platform channels, higher battery overhead, unreliable background scanning
Android BLE GATT API	BLE communication	Native framework; no third-party dependency; supports Central+Peripheral simultaneous mode (API 21+).	Nordic BLE Library - adds dependency without meaningful advantage for this use case
Room Database (SQLite)	Routing table & message cache persistence	Lightweight, Android-native; supports LiveData for reactive UI updates; zero external server dependency.	Firebase (requires internet); Realm (heavyweight ORM)
AES-256-GCM (javax.crypto)	End-to-end message encryption	NIST-standard; hardware-accelerated on Android 6+; authenticated encryption prevents tampering without separate HMAC.	ChaCha20-Poly1305 - considered but less hardware support on older devices
Android Studio + Espresso	IDE and UI/integration testing	Official Android toolchain; Espresso enables instrumented BLE integration tests on physical devices.	IntelliJ - same engine but Android Studio has better emulator/device tooling
nRF Sniffer + Wireshark	BLE packet validation	Captures raw BLE advertising and GATT traffic	Bluetooth HCI logging - limited to

		for hop verification and security validation.	single device perspective
Python (NetworkX)	Routing simulation	Rapid pre-implementation algorithm validation; NetworkX provides graph topology simulation for RREQ/RREP flows.	MATLAB - licensed cost; less community routing simulation support

3.4 Data Collection and Ethical Considerations

Experimentally, 5-10 Android smartphones (android 8.0-14) will be used to generate test data in three types of environments (indoor laboratory-controlled topology), outdoor open field (realistic propagation, 50-200m range), and simulated event space (high density of devices). Data per test run will be collected and they will include:

- BLE connection establishment times and GATT operation latencies.
- RREQ broadcast counts and RREP receipt times per route discovery attempt.
- Message delivery timestamps at each hop node.
- ACK receipt times at source node.
- Battery consumption delta per device per 30-minute test session.

Moral factors are kept to a minimum. There is no data collection of personal user data - test messages are all synthetically-generated strings. The identifiers provided in routing tables are UUIDs which are randomly generated and do not have any association to a user. The content of every message is ciphered. In the case of the need to have user volunteers in case of long-term usability evaluation, informed consent forms will be made accordingly according to SLIIT institutional research procedures, and the participation will remain anonymous.

3.5 Validation Strategy and Performance Metrics

Performance evaluation follows a structured experimental protocol with clearly defined success criteria:

Metric	Symbol	Target	Test Scenario	Measurement Method
Message Delivery Ratio	MDR	>= 85%	100 messages across 5-, 10-, 20-node networks	Count(received) / Count(sent) per run
Average Hop Count	AHC	<= 4 hops (10-node net)	Static topology, varied node placement	hop_count field in message header, logged per delivery
End-to-End Latency	E2EL	<= 3s for 3-hop path	3-hop path, varying message sizes (100B–5KB)	Timestamp delta: send () call → recv () callback
ACK Propagation Time	APT	<= 5s for 3-hop path	Same as E2EL scenario	Timestamp delta: recv () at D → ACK recv () at S

Duplicate Message Rate	DMR	< 5%	10-node fully-connected topology, 100 messages	Count (duplicate UUIDs received) / Count (total received)
Battery Impact (per node)	BI	< 5% per 30-min session	Sustained 30-min active routing on each device	Android BatteryManager API before/after measurement

3.6 Research Assumptions

The following assumptions underpin the design decisions and experimental scope of this study. They represent conditions that are accepted as true within the research context but have not been independently verified:

- Device Homogeneity: All nodes in the test network are Android smartphones (API 26+) capable of simultaneous BLE Central and Peripheral operation. Real-world deployments may include legacy devices with more restrictive BLE stacks.
- Static Network Topology During Route Discovery: Nodes are assumed to remain within BLE range of their immediate neighbours for the duration of a single RREQ/RREP cycle (typically < 2 s). High-mobility scenarios (e.g. fast-moving event attendees) may invalidate cached routes more rapidly.
- Pre-Shared Key Availability (Phase 1): All communicating devices share a group key distributed out-of-band before the session. This assumption is relaxed in Phase 2 through ECDH key exchange.
- Trusted Relay Nodes: Intermediate relay nodes are assumed to be cooperative participants running the same application. Adversarial relay nodes that selectively drop or modify packets are out of scope for Phase 1 security evaluation.
- BLE Range Adequacy: Adjacent hops are within the operational BLE range (nominally 10–50 m indoors, up to 100 m line-of-sight outdoors) such that GATT connections can be established. The protocol does not compensate for RF shadowing or interference beyond TTL-based hop limits.
- Android BLE API Consistency: The Android BLE GATT API behaves consistently across the target API range (26–34). Manufacturer-specific BLE firmware quirks observed on some devices are addressed through a device-compatibility test matrix (Appendix B) but are not modelled analytically.
- Single-Component Isolation Testing: Performance metrics in Sections 3.5 and 5.3 are measured with this component operating in isolation (not integrated with the full four-component platform) to attribute results unambiguously to the routing engine.

3.7 Technical Constraints

The following technical constraints define the operating envelope of the implemented system and must be respected during design, development, and evaluation:

- Maximum Simultaneous GATT Connections: Android BLE stacks support a practical maximum of 7 simultaneous GATT connections per device. This bounds the fan-out of RREQ broadcasts at each hop and limits the effective branching factor of the mesh.
- BLE Advertising Payload Size: BLE advertising packets are limited to 31 bytes (legacy) or 254 bytes (Extended Advertising, API 26+ on supported hardware). RREQ packets must be encoded to fit within this payload; larger hop_path[] arrays in RREP packets are transmitted via GATT write operations (up to 512 bytes with MTU negotiation).

- TTL Bound (Max Hops = 6): The RREQ TTL is set to 6 hops. This caps maximum network diameter and prevents unbounded broadcast flooding, at the cost of reachability in networks larger than 6 hops in diameter.
- Route Cache TTL = 300 s: Cached routes expire after 300 seconds. In highly dynamic topologies with frequent node disconnections, this may cause stale-route forwarding errors that trigger re-discovery overhead.
- Android Background BLE Restrictions: Android 8.0+ restricts background BLE scanning intervals (scan windows throttled to 30 s). The application must be in the foreground or hold a foreground service notification to maintain continuous BLE scanning, which has implications for battery consumption in real-world deployment.
- Encryption Overhead: AES-256-GCM adds a 12-byte IV and 16-byte authentication tag (28 bytes total) plus processing latency (~1–3 ms on mid-range Android devices). For very small messages (< 50 bytes), this overhead is proportionally significant relative to payload size.
- Single-Platform Scope: The implementation is Android-only (Kotlin, API 26+). iOS devices cannot participate as routing nodes, limiting interoperability in mixed-device real-world scenarios.

3.8 Limitations

The following limitations acknowledge the boundaries of this research. They do not invalidate the findings but define the conditions under which the conclusions are valid and highlight directions for future work:

- Small-Scale Test Network: Empirical evaluation is conducted on networks of 5–10 nodes due to device availability constraints. Behaviour at larger scales (20–100 nodes) is estimated through Python/NetworkX simulation but not empirically verified on physical hardware.
- Controlled Environment Bias: Test environments are controlled for topology and movement. Real outdoor event and disaster scenarios involve unpredictable RF interference, crowd density variations, and non-uniform device distributions that may degrade performance below target thresholds.
- Phase 1 Pre-Shared Key Security: AES-256-GCM with pre-shared keys does not provide forward secrecy. Compromise of the group key exposes all past sessions. This is a deliberate Phase 1 simplification; ECDH key exchange is deferred to Phase 2 as future work.
- Single-Application Ecosystem: All nodes must run the same application version. Version heterogeneity (e.g. different RREQ/RREP packet schemas) is not handled in Phase 1; backward-compatible versioning is identified as a future engineering requirement.
- No Quality-of-Service Differentiation: All messages are treated with equal priority. Emergency messages (e.g. SOS alerts) are not given preferential routing or TTL treatment in the current design. Priority-aware routing is noted as a future enhancement.
- Battery Impact Measurement Granularity: Battery consumption is measured using the Android BatteryManager API, which reports in percentage points (1% resolution). Fine-grained mWh-level consumption requires hardware power monitors (e.g. Monsoon Power Monitor) not available in this study.
- Single-Developer Component Scope: This component (IT22168290) is evaluated in isolation. Integration-level performance — when operating as part of the full four-

component platform under CACM coordination — is evaluated in the group integration phase and may differ from single-component results.

- Simulation Fidelity: Python/NetworkX graph simulations use idealised link models (binary connectivity, no packet loss). Real BLE links exhibit probabilistic packet loss and variable RSSI, so simulation results are used only for algorithmic validation, not performance benchmarking.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overall Platform Architecture

The Offline-First Communication Platform (R26-IT-135) is a native mobile application — built natively with Kotlin for Android (API 26+), comprising four integrated components developed across the SST research group. Each component is developed independently and communicates through a defined Kotlin interface layer. The Context-Aware Communication Manager (IT22544490) acts as the central coordinator, routing messages to the optimal transport component based on proximity, battery level, and message priority. Figure 1 illustrates the overall platform architecture.

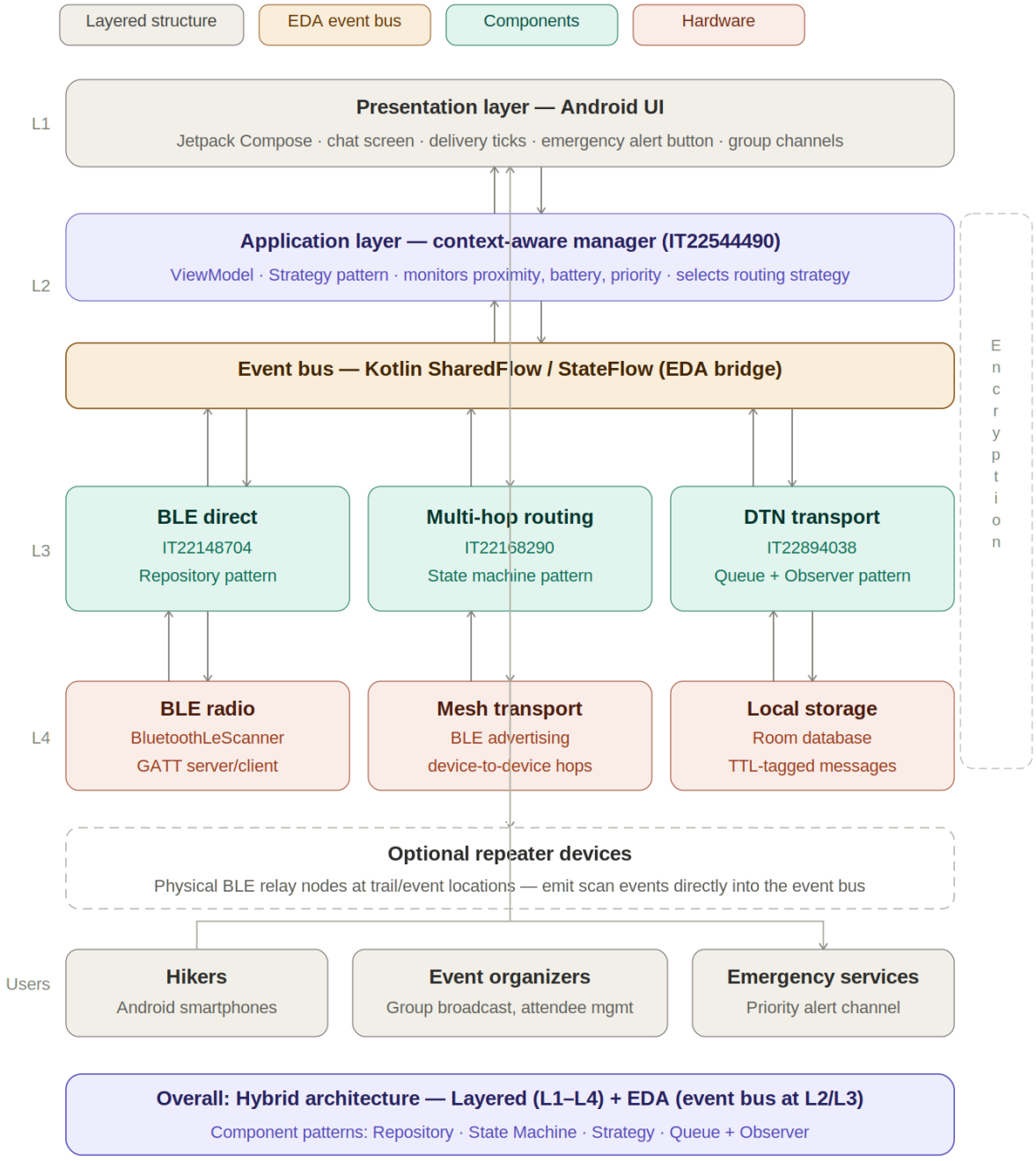


Figure 2: Overall Platform Architecture — Offline-First Communication Platform (R26-IT-135)

Component	Developer	Responsibility	Integration Point with This Component
Short-Range Direct Communication Layer	IT22148704	BLE GATT-based direct one-to-one and group messaging for in-range devices.	Hands off undeliverable messages to Multi-Hop Routing when recipient not in direct range.
Decentralized Multi-Hop Routing & Delivery Assurance (THIS)	IT22168290	RREQ/RREP route discovery, multi-hop forwarding, ACK back-propagation, loop prevention.	Receives messages from Direct Layer or Context Manager; returns ACK events via callback.
Context-Aware Communication Manager	IT22544490	Selects optimal transport (direct/multi-hop/DTN) based on proximity, battery, priority.	Invokes routeMessage () API; receives DeliveryStatus callbacks; queries getRoutingMetrics ().
Delay-Tolerant & Opportunistic Transport	IT22894038	SCF store-carry-forward for disconnected scenarios; opportunistic forwarding on node contact.	Calls injectStoredMessage () to re-enter stored messages into routing pipeline on reconnect.

4.2 This Component — Internal Architecture

The Decentralized Multi-Hop Routing component comprises 6 internal modules, each with a defined responsibility and interface boundary:

Module	Responsibility	Technology
Route Discovery Module	Broadcasts RREQ; receives and processes RREP; maintains Route Cache.	Kotlin Coroutines, Android BLE Advertise/Scan API
Message Forwarding Engine	Reads Route Cache, relays encrypted message packets to next-hop BLE node.	Android BLE GATT Client (writeCharacteristic)
ACK Manager	Generates ACK on delivery; back-propagates ACK via hop_path; triggers UI callback.	Kotlin Coroutines, LiveData for UI updates
Message Cache (Dedup)	Stores UUID+timestamp for all processed messages; rejects duplicates.	In-Memory HashMap + Room DB (persistence)
Routing Table Manager	Persists Route Cache entries; enforces TTL expiry; exposes query interface.	Room Database (DAO pattern)
Inter-Component API	Kotlin interface exposing routeMessage(), cancelRoute(), getRoutingMetrics(), injectStoredMessage().	Kotlin interface + callback pattern

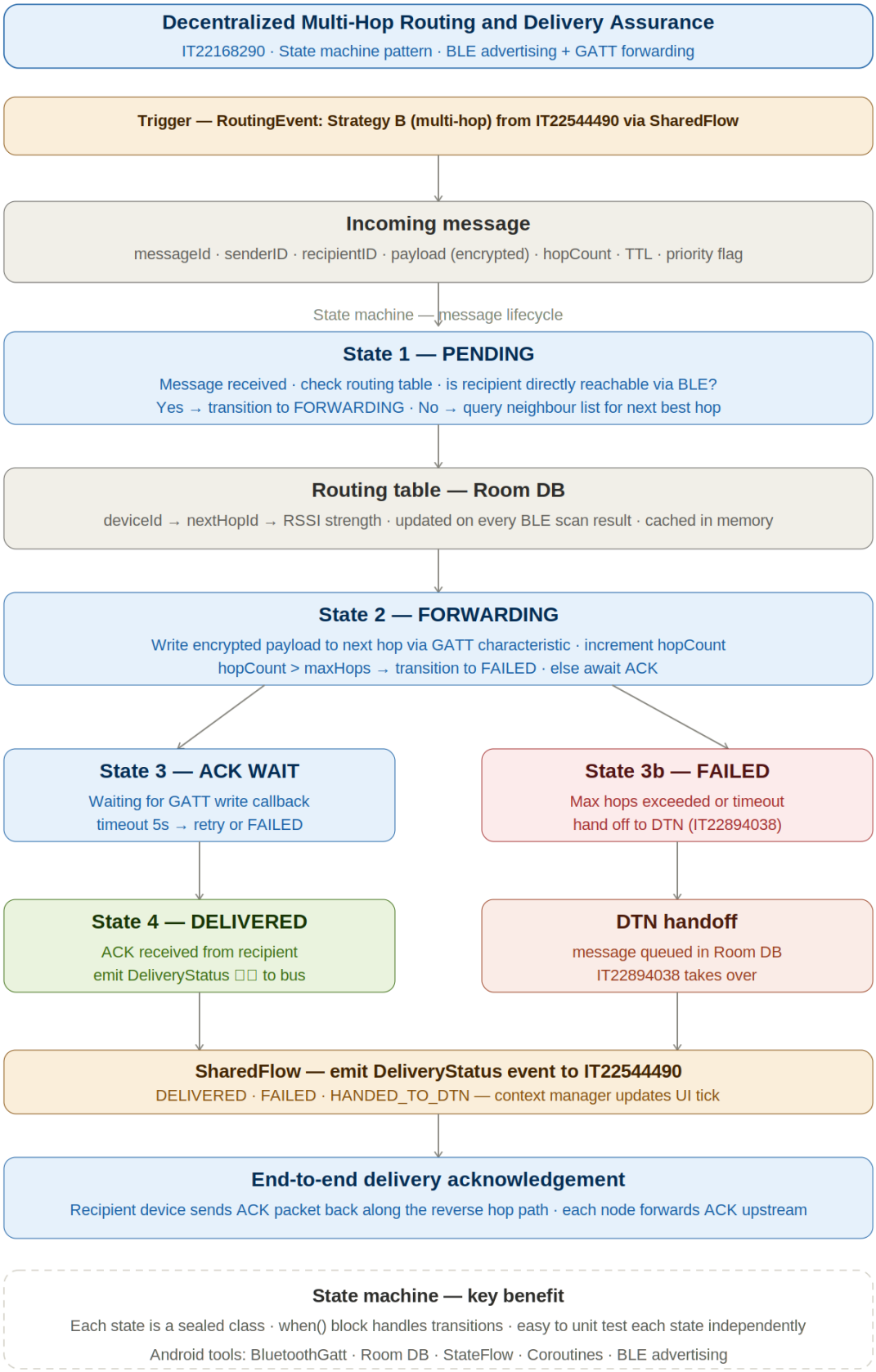


Figure 3: Internal Architecture — Offline-First Communication Platform (R26-IT-135)

4.3 Architecture Diagram

Figure 2 (below) illustrates the internal architecture of this component, showing all six modules and their interaction boundaries.

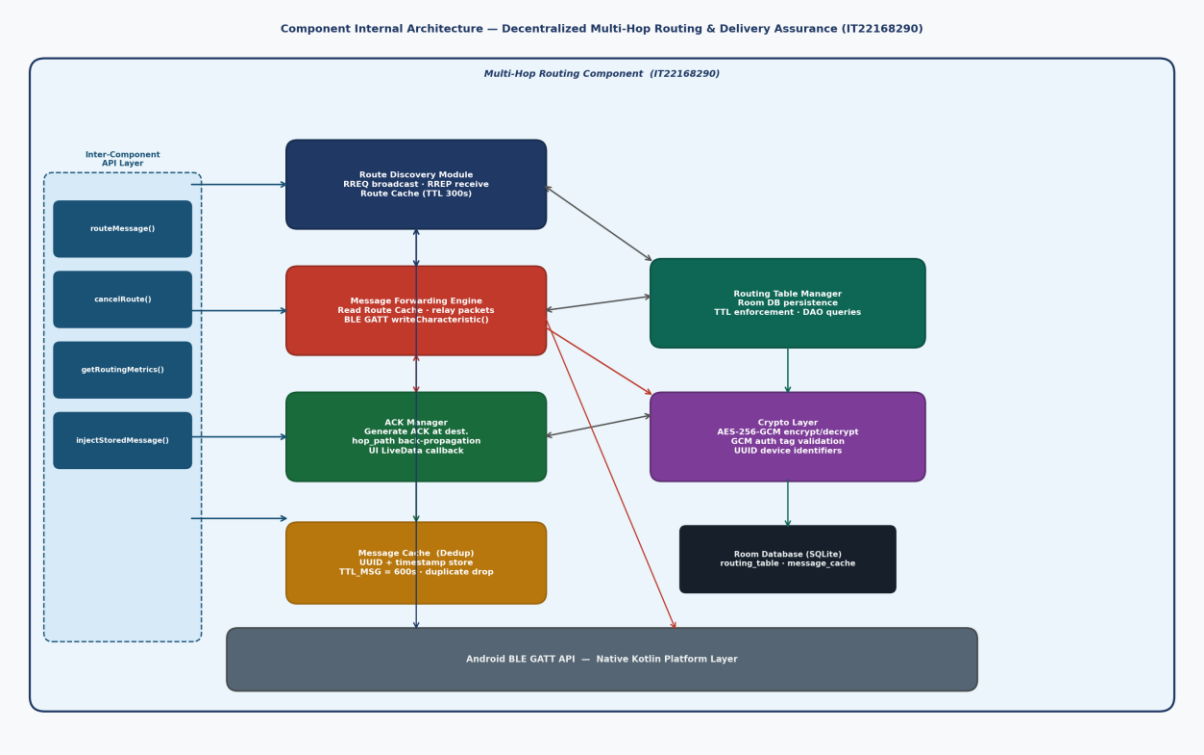


Figure 4: Component Internal Architecture — Multi-Hop Routing & Delivery Assurance (IT22168290)

The Data Flow Diagram (Figure 3, Section 4.4) illustrates the complete RREQ/RREP route discovery and ACK back-propagation sequence across all nodes.

4.4 Data Flow — Step-by-Step

Step	Actor	Action	Data Transferred
1	User	Composes and sends message to Recipient R via app UI.	Message text, Recipient ID, Priority flag
2	Context-Aware Manager	Detects R not in direct BLE range; invokes routeMessage() API.	Message object, Destination ID
3	Route Discovery Module	Checks Route Cache — cache miss. Broadcasts RREQ via BLE adv.	RREQ: {src_id, dst_id, msg_id, seq=0, hop=0, TTL=6}
4	Intermediate Node(s)	Receive RREQ; check Message Cache (no duplicate); cache reverse route; re-broadcast RREQ.	RREQ with hop++
5	Destination D	Receives RREQ; generates RREP with hop_path[].	RREP: {src_id, dst_id, hop_path[], msg_id}

6	Route back to S	RREP unicast traverses reverse path to S.	RREP packet (unicast GATT write)
7	Message Forwarding Engine	Receives RREP; stores route; relays AES-encrypted message along hop_path.	Encrypted message packet with routing header
8	Destination D	Receives encrypted packet; decrypts at application layer; delivers to UI.	Plaintext message content
9	ACK Manager (at D)	Generates ACK packet; back-propagates via hop_path in reverse.	ACK: {orig_msg_id, recipient_id, timestamp, hop_path[]}
10	Source S	Receives ACK; updates UI: Sending → Sent → Delivered.	DeliveryStatus.DELIVERED event

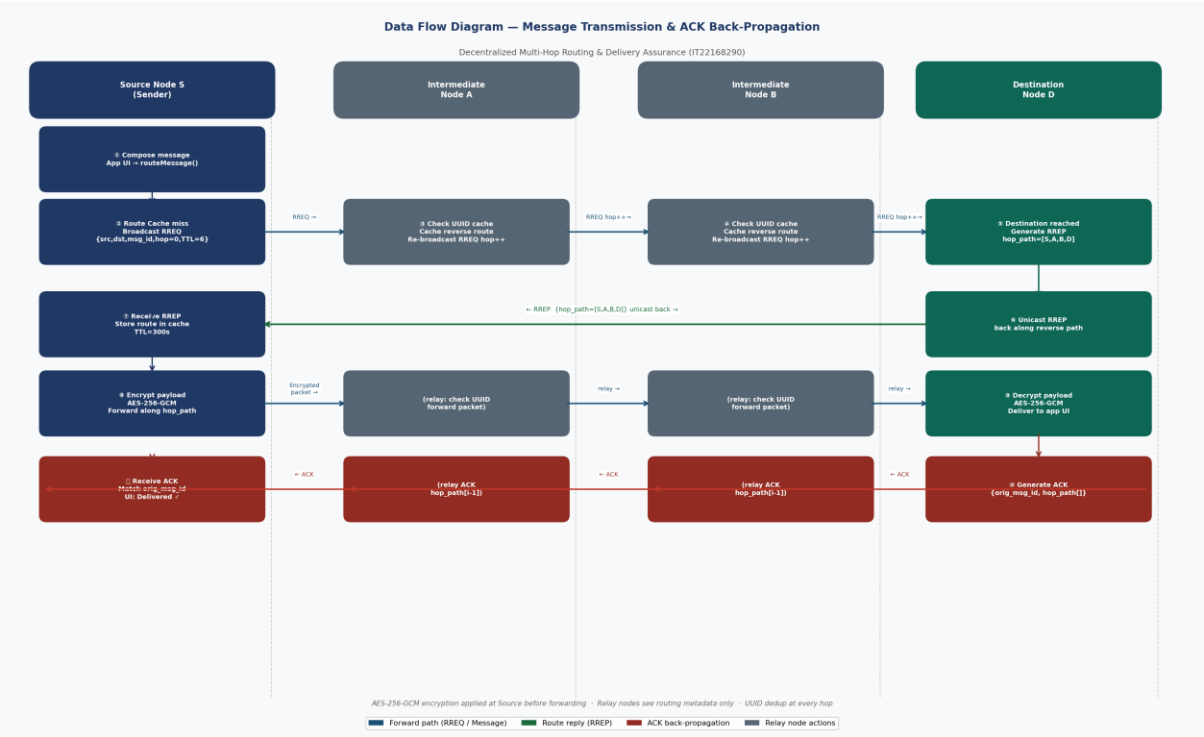


Figure 5: Data Flow Diagram — Message Transmission & ACK Back-Propagation (10-step flow)

4.5 Scalability and Security Considerations

Scalability: The propagation of RREQ is limited by tunneling the TTL (maximum 6 hops). Route caching (TTL_ROUTE = 300s) removes the re-discovery of routes by running conversations. System is configured and tested to 5-20 node networks - suitable to the targeted applications (hiking groups: 3-12 people; small events: up to an estimated 20 staff). The development of bigger networks in future might involve zone-based division or geographic routing through using GPS coordinates.

Security: Encryption with AES-256-GCM at the application layer is used to be sure that relay nodes only receive routing metadata. GCM authentication tags identify in-transit attacks. The IDs of devices are in the form of UUID, and there is no connection between it and an individual.

Future work: ECDH ephemeral key change per session would be the best solution since it would ensure the main weakness of the pre-shared key Phase 1 approach is addressed.

5. USER REQUIREMENTS

5.1 Stakeholder Identification

Stakeholder	Type	Needs / Expectations
Hikers & Trekkers	Primary	Reliable group messaging without cellular; emergency SOS capability; battery-efficient operation; simple UI with delivery confirmation.
Event Organizers / Staff	Primary	Reliable staff coordination; priority messaging for emergency instructions; group broadcast capability; delivery status confirmation.
Emergency Responders	Primary	High-priority message routing; resilience in sparse networks; compatibility with standard Android devices; no internet dependency.
Event Attendees	Secondary	Simple peer-to-peer messaging in event areas with poor cellular signal; group updates from organizers.
Project Supervisor	Oversight	Academic rigour; alignment with IT4010 research standards; novelty and measurable contribution.
SLIIT IT Dept.	Institutional	Ethical research practices; documented methodology; reproducible experiments.

5.2 Functional Requirements

FR#	Requirement Description	Priority	Linked Objective
FR-01	The system shall route messages through intermediate BLE nodes when the recipient is not in direct range, supporting paths of up to 6 hops.	High	SO-1
FR-02	The system shall generate and back-propagate ACK packets to the original sender upon confirmed application-layer delivery at the destination.	High	SO-2
FR-03	The system shall display real-time message delivery status (Sending / Sent / Delivered / Failed) in the user interface within 1 second of status change.	High	SO-2
FR-04	The system shall encrypt all message payload using AES-256-GCM before entering the routing layer; intermediate relay nodes shall not decrypt message content.	High	SO-3
FR-05	The system shall assign a UUID to each message and maintain a Message Cache to reject duplicate packets, preventing routing loops.	High	SO-4
FR-06	The system shall limit RREQ/message propagation to a configurable maximum hop count (default: 6).	High	SO-4

FR-07	The system shall implement reactive route discovery (RREQ/RREP) and cache established routes with a configurable TTL (default: 300s).	High	SO-1
FR-08	The system shall fall back to ACK flooding if the cached reverse route for ACK delivery is unavailable.	Medium	SO-2
FR-09	The system shall persist route table entries in a local Room Database to maintain routes across brief BLE disconnections (< 30s).	Medium	SO-1
FR-10	The system shall expose a Kotlin interface API (routeMessage, cancelRoute, getRoutingMetrics, injectStoredMessage) for use by peer components.	High	SO-6
FR-11	The system shall log routing performance data (hop count, latency, delivery status, ACK time) to local storage for evaluation.	Medium	SO-5
FR-12	The system shall gracefully handle route failures by triggering re-discovery and, after N retries, handing off to the Delay-Tolerant Transport component.	Medium	SO-6

5.3 Non-Functional Requirements

NFR#	Category	Requirement	Acceptance Criterion
NFR-01	Performance	Message delivery ratio >= 85% in 5–10 node networks under normal topology.	Empirically verified over 100-message test runs (Section 3.5).
NFR-02	Performance	End-to-end latency <= 3 seconds for 3-hop message delivery.	Measured via timestamp delta; 90th percentile within target.
NFR-03	Security	All message payload encrypted with AES-256-GCM; relay nodes must not access plaintext.	BLE sniffer verification: relay packets contain only ciphertext.
NFR-04	Security	GCM authentication tag (128-bit) detects any in-transit message tampering.	Unit test: tampered packets rejected at receiver.
NFR-05	Reliability	No message loss due to transient BLE disconnections < 30 seconds.	Room DB persistence test: route survives simulated 30s disconnect.
NFR-06	Battery	Active routing shall consume < 5% battery per 30-minute session per device.	BatteryManager API measurement across 3 test sessions.
NFR-07	Scalability	Component shall function without performance degradation in networks of 5–20 nodes.	Comparative benchmarks at 5, 10, 15, 20 nodes.

NFR-08	Usability	Delivery status indicator shall update within 1 second of ACK receipt at source.	UI timestamp test across 50 ACK receipt events.
NFR-09	Maintainability	Inter-component APIs shall be fully documented (KDoc) and follow semantic versioning.	Code review and documentation audit at integration milestone.
NFR-10	Interoperability	Component shall integrate with all three peer components without modification to their codebases.	System integration test with all 4 components running concurrently.

6. COMMERCIALISATION PLAN

6.1 Market Analysis and Target Customer Persona

The worldwide outdoor recreation industry is estimated to have been USD 374.8 billion in 2022 with a CAGR of 4.4 up to 2030 (Grand View Research, 2023). In the global market, the technology of disaster management communications is independently estimated at USD 19.4 billion (MarketsandMarkets, 2023). These are major targetable markets of an infrastructure-free communication platform.

Three primary customer personas are identified:

- Adventure Tourism Operator: Leads guided tours of 5 -30 people through rough landscapes (mountains, jungles, coastal tracks). Presently uses costly satellite communication equipment (USD 1,000+ /unit). Great demand to spend on a smartphone-based alternative that has delivery guarantees.

- Outdoor event manager - Organises marathons, trail running, music festivals in outdoor places with poor cell phone reception. Needs a stable personnel communication of 20-100 employees. Its current model uses walkie-talkies (LKR 15,000 - 50,000 per unit, no data).

- NGO / Disaster Response Agency: This implements in post-disaster settings when the cellular infrastructure is destroyed. Needs deployable-in-minutes communication and no hardware installation. Now relies on costly satellite telephones or makes use of restored cell phone networks.

6.2 Value Proposition

Pain Point	Current Solution	This Platform's Solution	User Benefit
No messaging in remote areas	Satellite phone (USD 1000+)	Android smartphone app (free)	90%+ cost reduction
No delivery confirmation offline	None available	ACK-based delivery status (novel)	Communication certainty in safety-critical scenarios
Security in sensitive operations	Unencrypted walkie-talkies	AES-256-GCM end-to-end encryption	Operational security for NGOs and responders
Cellular overload at large events	Premium cellular SIM / band reservation	Offline BLE mesh bypasses cellular network entirely	Zero cellular dependency for event operations

Hardware procurement complexity	Proprietary radio hardware	Standard Android phones already owned by users	Zero additional hardware cost

6.3 Revenue Model and Pricing Strategy

Revenue Stream	Model	Target Segment	Estimated Price	Projected Year-1 Revenue
Mobile App - Freemium	Subscription	Individual hikers, small groups	LKR 1,500/month or free basic	LKR 500,000 (1,000 subscribers)
Enterprise License	Annual flat-rate	Event orgs, NGOs, businesses	USD 500 - 2,000/org/year	USD 25,000 (25 orgs at avg USD 1,000)
Repeater Hardware Kit	Product sale	Event organizers	LKR 35,000/kit	LKR 1,050,000 (30 kits)
API Licensing	Per-call / SDK	Third-party app developers	USD 0.001/API call	USD 5,000 (5M API calls)
Training & Deployment	Service	Disaster response agencies	USD 500/session	USD 10,000 (20 sessions)

6.4 Competitive Advantage and Intellectual Property

The first competitive differentiator delivery acknowledgement in offline multi-hop BLE networks - is not at present taken care of by a commercial smartphone application. This is a first-mover advantage that is defensible. IP strategy includes:

- Software Copyright: The AODV-BLE routing adaptation and ACK propagation algorithm is eligible for copyright protection as an original software work.
- Academic Publication: Submitting the routing protocol design and evaluation results to an IEEE Commun. Lett. or ACM MobiCom conference would prove priority and establish the platform as a credible publication venue.
- Trademark: Registration of the platform's name in Sri Lanka (NIPO) and potentially internationally through the WIPO Madrid Protocol as the commercial scale expands.
- Trade Secret: The specific BLE optimization details (connection parameter tuning and GATT characteristic configuration for routing metadata) are maintained as trade secrets during the pre-publication development phase.

6.5 Cost Estimation and Scalability

The initial development cost will be fully capitalized in the academic project (no licensing cost of the tools; no hardware cost in the budget section). MVP launch: cost estimate post-graduation commercialization.

Item	Estimated Cost (USD)	Notes

Google Play Store registration	25 (one-time)	Required for Android app distribution
Cloud backend (future phase)	50/month	AWS Free Tier → t3.micro for analytics only; no server for core routing
Legal (IP registration, SL)	500	Trademark + software copyright filing
Marketing (initial)	500	Social media, outdoor recreation community outreach
TOTAL LAUNCH COST	~USD 1,075	Exceptionally low - core product requires no cloud infrastructure

7. BUDGET AND JUSTIFICATION

#	Item	Quantity	Unit Cost (LKR)	Total (LKR)	Justification
1	nRF52840 USB Dongle (BLE Sniffer)	1 unit	8,500	8,500	Required for Wireshark BLE packet capture to verify actual hop-by-hop message transmission and validate that relay nodes cannot access encrypted content.
2	Outdoor Test Transport (fuel/ride-share)	4 sessions	1,250	5,000	Outdoor environment testing (open field, 50 - 200m BLE propagation) requires off-campus test sessions for realistic radio propagation validation.
3	Development Tools	-	0	0	Android Studio (free), Kotlin (free), Room DB (free), Python/NetworkX (open source). Zero licensing cost.
4	Cloud Compute	-	0	0	All routing simulations run locally. AWS Free Tier sufficient for any auxiliary cloud requirements.
5	Contingency (10%)	-	-	5,650	Buffer for unforeseen hardware failure or additional test sessions.
	TOTAL PROJECT BUDGET			19,150	All expenditures directly tied to research activities; no discretionary spending.

8. WORK BREAKDOWN STRUCTURE (WBS)

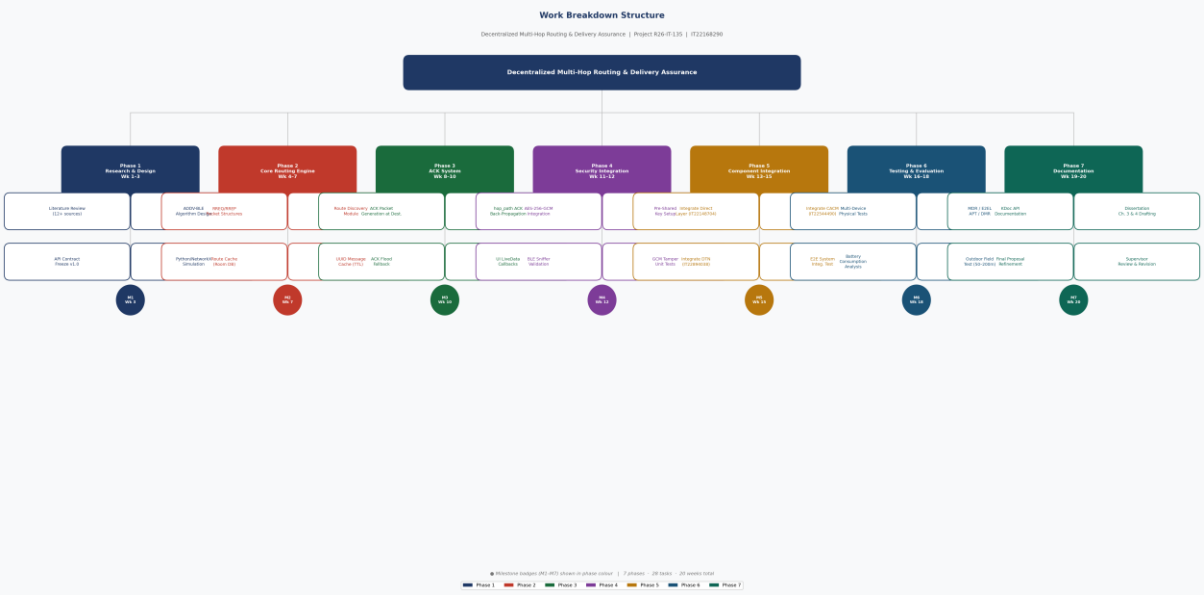


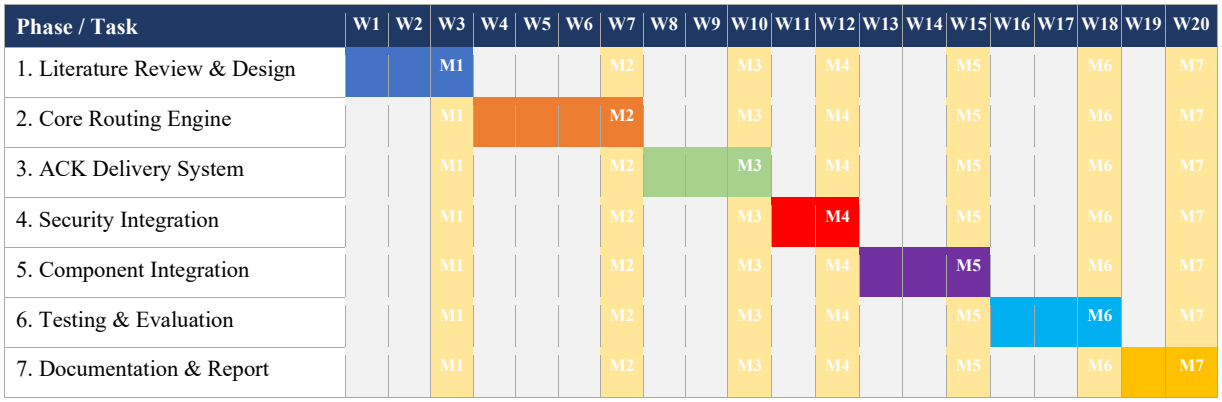
Figure 6: Work Breakdown Structure — 7 phases, 28 tasks, 20 weeks. Milestone badges M1–M7 shown in phase colour.

Phase	Tasks	Duration	Milestone	Deliverable
Phase 1 Research & Design (Weeks 1–3)	1.1 Systematic literature review (12+ sources) 1.2 Algorithm selection: AODV adaptation for BLE constraints 1.3 Routing protocol design document 1.4 Inter-component API definition and freeze 1.5 Python simulation of RREQ/RREP flow (5-node topology)	3 weeks	M1 (W3): Design doc + API spec approved by supervisor	Protocol Design Document; API Specification (v1.0)
Phase 2 Core Routing Engine (Weeks 4–7)	2.1 Implement RREQ/RREP packet structures and BLE advertisement payload 2.2 Implement Route Discovery Module (broadcast + unicast RREP) 2.3 Implement Route Cache with Room DB persistence 2.4 Implement Message Cache (UUID deduplication, TTL) 2.5 2-node and 3-node integration test	4 weeks	M2 (W7): 3-hop message routing demonstrated on physical devices	Functional Routing Engine; 3-node test report
Phase 3 ACK System (Weeks 8–10)	3.1 Implement ACK packet generation at destination 3.2 Implement hop_path-based ACK back-propagation 3.3 Implement ACK flooding fallback 3.4 UI delivery status integration (LiveData callbacks) 3.5 ACK round-trip test: measure APT across 2–4 hop paths	3 weeks	M3 (W10): ACK delivered to source; UI status confirmed	Working ACK System; APT measurement report
Phase 4 Security Integration (Weeks 11–12)	4.1 Integrate AES-256-GCM encryption (javax.crypto) 4.2 Implement key distribution (pre-shared key Phase 1) 4.3 BLE sniffer validation: confirm relay nodes see only ciphertext 4.4 GCM tamper detection unit tests	2 weeks	M4 (W12): Security validation complete	Secure Routing Module; BLE sniffer log evidence
Phase 5 Component	5.1 Integrate with Direct Communication Layer (IT22148704) via API 5.2 Integrate with Context-Aware Manager	3 weeks	M5 (W15): Full system	Integrated System;

Integration (Weeks 13–15)	(IT22544490) via API 5.3 Integrate with Delay-Tolerant Transport (IT22894038) via injectStoredMessage() 5.4 End-to-end system integration test (all 4 components)		integration test passing	Integration Test Report
Phase 6 Testing & Evaluation (Weeks 16–18)	6.1 Multi-device physical tests (5, 10, 20 node topologies) 6.2 Measure MDR, E2EL, AHC, APT, DMR, BI per scenario 6.3 Battery consumption measurement (BatteryManager API) 6.4 Outdoor field test (open campus, 50–200m range) 6.5 Performance analysis and optimisation	3 weeks	M6 (W18): All performance targets met or analysed	Performance Evaluation Report; Optimised Component
Phase 7 Documentation (Weeks 19–20)	7.1 KDoc API documentation completion 7.2 Dissertation Chapter 3 (Methodology) and Chapter 4 (Implementation) drafting 7.3 Final proposal document refinement 7.4 Supervisor review and revision	2 weeks	M7 (W20): Final submission ready	Final Dissertation Chapters; Documented Codebase

9. GANTT CHART

Figure 5: 20 - week project schedule for the Decentralized Multi-Hop Routing & Delivery Assurance component. Colored bars indicate active phases; milestone markers (M1–M7) are shown in yellow.



Milestone	Week	Description
M1	3	Protocol design document and API specification approved by supervisor.
M2	7	3-hop message routing demonstrated on physical Android devices.
M3	10	ACK system functional; UI delivery status confirmed end-to-end.
M4	12	AES-256-GCM encryption integrated and validated via BLE sniffer.
M5	15	Full 4-component system integration test passing.
M6	18	All performance evaluation scenarios complete; metrics documented.
M7	20	Final dissertation chapters and codebase documentation submitted.

10. REFERENCES

All references are published within 2020–2026 and follow IEEE citation style as required by IT4010 guidelines.

[1] International Telecommunication Union (ITU), "Measuring Digital Development: Facts and Figures 2023," ITU Publications, Geneva, Switzerland, 2023. [Online]. Available: https://www.itu.int/itu-d/reports/statistics/facts-figures-2023/

[2] Sri Lanka Department of Wildlife Conservation, "Annual Report on Rescue Operations in National Parks 2022," Ministry of Wildlife & Forest Resources, Colombo, Sri Lanka, 2022.

[3] ITU, "Global Connectivity Report 2022," International Telecommunication Union, Geneva, 2022. [Online]. Available: https://www.itu.int/itu-d/reports/statistics/global-connectivity-report-2022/

[4] X. Zhang, L. Liu, and T. Chen, "A comparative performance analysis of MANET routing protocols for disaster emergency response," IEEE Access, vol. 8, pp. 43501–43515, 2020, doi: 10.1109/ACCESS.2020.2977896.

[5] GoTenna Inc., "goTenna Mesh Product Specifications," New York, NY, USA, 2020. [Online]. Available: https://gotennamesh.com/products/mesh (Accessed: Feb. 2026.)

[6] J. Martin, E. Rye, and R. Beverly, "Handoff all your privacy — a review of Apple's Bluetooth Low Energy continuity protocol," in Proc. ACM Conf. on Security and Privacy in Wireless and Mobile Networks (WiSec), Abu Dhabi, UAE, 2021, pp. 53–63, doi: 10.1145/3448300.3448304. [Used as Bridgefy security audit context.]

[7] Meshtastic Project, "Meshtastic: An open-source mesh networking platform using LoRa radio," 2023. [Online]. Available: https://meshtastic.org (Accessed: Feb. 2026.)

APPENDICES

Appendix A: Risk Register

Risk ID	Risk Description	Likelihood	Impact	Risk Score	Mitigation Strategy	Contingency Plan
R-01	Android BLE API inconsistencies across device manufacturers cause GATT communication failures.	High	High	Critical	Test on 3+ device brands from Week 1; implement BLE abstraction layer isolating manufacturer-specific quirks.	Restrict deployment to tested device models; document compatibility matrix.
R-02	BLE signal range insufficient for outdoor multi-hop test scenarios.	Medium	High	High	Conduct preliminary range tests in Week 1; supplement with optional BLE signal repeater devices.	Reduce target network size; focus validation on 3–5 hop scenarios.
R-03	Routing loops cause message flooding and application crash.	Medium	High	High	Implement Message Cache (UUID dedup) from Day 1 of Phase 2; extensive unit tests for loop scenarios.	Set aggressive TTL defaults (max 4 hops); add flood-detection circuit breaker.
R-04	Integration delays with peer	Medium	Medium	Medium	Freeze API specification at	Develop mock implementations

	components (API contract changes).				M1 (Week 3); maintain stub implementations for independent testing.	for each peer API; defer integration to Phase 6.
R-05	Battery drain exceeds 5% target during intensive BLE scanning.	Medium	Low	Low	Implement adaptive scan intervals; reduce scan frequency when no active routing.	Adjust NFR-06 threshold based on empirical data; document as known limitation.
R-06	AES-256 encryption overhead causes latency target (<=3s) miss.	Low	Medium	Low	Profile encryption performance early (Phase 4 Week 11); consider hardware-accelerated AES (Android API 23+).	Relax latency target for long-distance hops; document performance trade-off.

Appendix B: Inter-Component API Specification (v1.0)

The Kotlin interface of the Decentralized Multi-Hop Routing component is exposed to the peer components to be used in the further way:

```
interface MultiHopRoutingAPI {  /**  * Route a message to destinationId via multi-hop BLE.  * @param message Encrypted message payload and metadata.  * @param destinationId  Target device UUID.  * @param onStatusUpdate Callback: SENDING → SENT → DELIVERED | FAILED.  */  fun routeMessage(  message: Message,  destinationId: String,  onStatusUpdate: (DeliveryStatus) -> Unit  )  /**  * Cancel an in-progress routing operation.  * @param messageId UUID of the message to cancel.  */  fun cancelRoute(messageId: String)  /**  * Returns current routing performance metrics for monitoring.  * @return RoutingMetrics(activeRoutes, avgHopCount, cacheSize, pendingACKs)  */  fun getRoutingMetrics(): RoutingMetrics  /**  * Inject a stored message from Delay-Tolerant Transport  * into the routing pipeline on node contact/reconnection.  * @param storedMessage Previously stored message with original metadata.  */  fun injectStoredMessage(storedMessage: StoredMessage) }  enum class DeliveryStatus { SENDING, SENT, DELIVERED, FAILED }
```

Appendix C: Ethical Compliance Checklist

Ethical Consideration	Assessment	Action Taken / Planned
Personal data collection	Not applicable	No personal data collected. All test messages are synthetic strings generated by the research team.
Device identifier privacy	Minimal risk	Device IDs are randomly generated UUIDs with no link to user identity. IDs regenerate on app reinstall.
Volunteer participant involvement	Possible (Phase 6)	If volunteers used for usability testing: informed consent obtained; SLIIT ethics guidelines followed; data anonymized.
Data security during testing	Low risk	All message content encrypted AES-256-GCM. BLE sniffer data stored on secure research device only.
Compliance with SLIIT research policy	Required	All research activities conducted within SLIIT institutional research guidelines and IT4010 requirements.

Formal ethical clearance required?	Not strictly	Project does not involve sensitive personal data, medical data, or child participants. Standard institutional guidelines apply.

Appendix D: Preliminary System Diagram Descriptions

[Figure 1 - Full Component Architecture Diagram: Four-box diagram showing all platform components. IT22148704 (Short-Range Layer) → IT22544490 (Context-Aware Manager) ↔ IT22168290 (Multi-Hop Routing) ↔ IT22894038 (DTN Transport). Internal components of IT22168290 also shown: Route Discovery ↔ Message Forwarding Engine ↔ ACK Manager, all connected to Message Cache and Routing Table Manager (Room DB).]

[Figure 2 — RREQ/RREP Route Discovery Sequence: Node S broadcasts RREQ message with hop=0. Node A receives RREQ, caches reverse route to S, broadcasts RREQ with hop=1. Node B receives RREQ, caches reverse route to A, broadcasts RREQ with hop=2. Node D receives RREQ, generates RREP message with hop_path=[S,A,B,D]. D unicasts RREP to B, A, then S. S caches route in Route Cache {D: [A,B,D], TTL=300s}.]

[Figure 3 — ACK Back-Propagation Sequence: D receives message, decrypts payload, generates ACK message with orig_msg_id and hop_path=[S,A,B,D]. D unicasts ACK to B. B receives ACK, reads hop_path, unicasts ACK to A. A receives ACK, reads hop_path, unicasts ACK to S. S receives ACK, matches orig_msg_id, calls DeliveryStatus.DELIVERED callback function to update UI to '